

Software Requirements Specification Template

Movie Ticket Booking Project

Software Requirements Specification

<1.2>

10/22/2026

Group 8

Leah Bernstein, Ows Zoabel, Hsa Say

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2023

Revision History

Date	Description	Author	Comments
<date>	<Version 1>	<Your Name>	<First Revision>
09/24/2026	1.0	Team	Rough Draft (Sections 1-3.5)
10/08/2026	1.1	Team	Software Design Specification
10/22/2026	1.2	Team	SDS Test Plan

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

REVISION HISTORY ..... **II**

DOCUMENT APPROVAL ..... **II**

1. INTRODUCTION **1**

1.1 PURPOSE 1
1.2 SCOPE 1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 1
1.4 REFERENCES 1
1.5 OVERVIEW 1

2. GENERAL DESCRIPTION **2**

2.1 PRODUCT PERSPECTIVE 2
2.2 PRODUCT FUNCTIONS 2
2.3 USER CHARACTERISTICS 2
2.4 GENERAL CONSTRAINTS 2
2.5 ASSUMPTIONS AND DEPENDENCIES 2

3. SPECIFIC REQUIREMENTS **2**

3.1 EXTERNAL INTERFACE REQUIREMENTS 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 FUNCTIONAL REQUIREMENTS 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 USE CASES 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 CLASSES / OBJECTS 3
3.4.1 <Class / Object #1> *3*
3.4.2 <Class / Object #2> *3*
3.5 NON-FUNCTIONAL REQUIREMENTS 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 INVERSE REQUIREMENTS 4
3.7 DESIGN CONSTRAINTS 4
3.8 LOGICAL DATABASE REQUIREMENTS 4
3.9 OTHER REQUIREMENTS 4

4. ANALYSIS MODELS **4**

4.1 SEQUENCE DIAGRAMS 5
4.3 DATA FLOW DIAGRAMS (DFD) 5
4.2 STATE-TRANSITION DIAGRAMS (STD) 5

5. CHANGE MANAGEMENT PROCESS **5**

A. APPENDICES **5**

A.1 APPENDIX 1 5
A.2 APPENDIX 2 5

1. Introduction

1.1 Purpose

The purpose of this document is to outline the requirements for developing a movie theater ticketing website. It serves as a guide for the system's expected functionality, user needs, and constraints. This document is intended for the development and other individuals involved in the design of the system.

1.2 Scope

Waverly Pictures is a movie theater ticketing website designed to be an optional online alternative for buying movie tickets. It is a completely distal system meant for online use only. Customers will not be required to purchase tickets in person or receive physical tickets. The website allows customers to browse available movies, view showtimes, select seats, and purchase tickets online. The website will support customer accounts, subscriptions, purchase history, and feedback. Its main goal is to create a simple and convenient experience for customers purchasing movie tickets through a web browser.

1.3 Definitions, Acronyms, and Abbreviations

SRS – Software Requirements Specification. A document which describes the requirements of the software system.

DDS – Design Document Specification. A document which describes the software architecture, logical design, and physical design of the system.

UAT – User Acceptance Testing. Testing carried out in order to ensure that the system satisfies the needs of the customer or end-user.

UI – User Interface. They are the visual elements of the website that are made up of buttons, menu, lists of movies and seat selection.

HTML – Hypertext Markup Language. HTML is a markup language that can be used to create websites.

CSS – Cascading Style Sheets. It is a technology that is used to control the look of the website.

Programming Language: JavaScript - It enables the website to function.

Database - Technology used for storage of data. The type of data includes customer information, movies, movie timings, seating, and buying records.

Software Architecture – The general structure of the software system along with the way its components interact with each other.

Functional Requirement – A function that the system should provide to its users.

Subscription – An extra paid service provided to customers who want to benefit from Waverly Pictures.

Payment Timer – It is a period of five minutes given to the customer to pay for their tickets.

End-User – The customer who uses the Waverly Pictures website to view movies, select seats, and purchase tickets.

Showtime – The date and time on which the movie becomes available to be watched at Waverly Pictures.

Seat Selection – The procedure of selecting available seats for viewing a particular movie on its selected showtime.

Purchase History – Details about the purchase of transactions and tickets purchased by a particular customer via the website.

Feedback – Information submitted by the customer after making a purchase on the site.

Authentication/Login – Procedure performed to identify a user account before gaining access to it.

Security – Features implemented to ensure the safety of customer accounts, personal information, and transaction details.

Integration Testing – Type of testing aimed at ensuring proper cooperation of various software system components and their interfaces.

Load Testing – Type of testing performed to check whether the system can operate with the anticipated load.

Bug – It is the fault in the program, which makes the system behave incorrectly.

Maintenance – The process of improving the software after its development and implementation.

1.4 References

The following resources will be used for developing the Waverly Pictures movie theatre website ticket booking system:

Course material and instructor instructions.
Software Requirements Specification (SRS) and requirements for the course project.
HTML manual for making the website.
CSS documentation on designing the appearance and structure of the website.
JavaScript documentation on making interactions on the website.

Database documentation and course materials on managing customer accounts, movies, showtimes, seating and ticket bookings.
Design Document Specification (DDS) guidance on building software architecture and designing the software.

1.5 Overview

This SRS document contains the requirements for the Waverly Pictures movie ticketing website. It contains the functional requirements of the system, like user accounts, user memberships, film information, movie showtimes, seat selection, ticket purchases, user feedback, and payment security. This document will also describe the system interface, server requirements, and constraints that will guide the development of the website.

2. General Description

2.1 Product Perspective

Waverly Pictures is a standalone online movie ticketing website. It will operate independently through a web browser and does not function as a component of another larger software system. It connects to the internet to display current movies, showtimes from nearby theaters in San Diego. It is not meant as a replacement for traditional movie booking but as a reliable alternative.

2.2 Product Functions

The functions of Waverly Pictures will support customers through the entire ticket-buying process.

Key features include:
Online platform allowing users to purchase tickets anywhere
Movies: Browse or search though the latest movies released
Availability: View available showtimes for nearby theaters
Seating: Select assigned seats using an interactive map
Buy Tickets: Transactions are secured.
Get digital receipts: Receive instant booking confirmation sent directly to an email or phone via text message

2.3 User Characteristics

The target audience for using the Waverly Pictures Website is the consumers wishing to book movie tickets on the website. The system is user-friendly for general users with some knowledge about computers and using web browsers. It is expected that users will be able to use websites, look up movies, check show times, choose their seats, and pay online.

The customers will have the ability to register and sign in and use some of the features including the customer's purchase history and the subscription. Customers will also give their feedback once they make a purchase of the ticket. There will not be any physical tickets, but email or text notifications.

No advanced technical expertise is required by the user of the website. The interface will be created in such a way that it becomes very easy to use. The user will just have to possess a desktop computer, a web browser, and an internet connection to use the system.

This system can be accessed by consumers of varying computer literacy. The website must thus contain clear guidelines and simple navigational steps in the purchase of tickets.

2.4 General Constraints

The General Constraints for the Waverly Pictures Website is a web application. The user system must be equipped with a web browser, and the user must have general knowledge of basic computer skills.

The system must be able to support online ticket purchases, and the database must be updated in real time to correctly support ticket availability.

The system must use a single database for all the theaters that is partitioned to support all 10 theaters across the San Diego area.

The system must support English, Spanish, Persian, and Arabic as well as the conversion of ticket prices into the user's local currency.

The system must be able to support at least 1000 concurrent users without performance degradation, with the ability to scale up to 10 million.

Each customer must be able to purchase up to 5 tickets in a single translation ranging from 2 weeks before 10 minutes after the scheduled movie showing.

Each customer is allowed a 10-minute purchasing window where the system temporarily holds the selected tickets.

The system must be able to distinguish between regular (150 seats) and deluxe theaters (75 seats).

The system must be able to handle student, military, and senior discounts when purchasing tickets.

The system must support payment processing through accepted payment methods including Visa, PayPal, and Apple Pay.

The system must include a customer feedback system that allows customers to submit feedback after completing a purchase.

The system must prevent or automated bots and excessive automated ticket purchases for high demand movies.

The system must restrict a customer account to being logged in on only one device concurrently.

The system must provide information about the number of tickets sold for each movie and the cost per ticket as well as daily logs of ticket purchases for record keeping.

The system must also be capable of displaying movie review scores and critic quotes from external review sources.

The system must allow users with loyalty accounts to store personal information, payment information, purchase history, and loyalty points. Tickets may only be returned or exchanged before the scheduled showing and must have been purchased through a registered user account.

Each ticket must be unique and non-replicable.

The system should be developed within 4 months with an initial budget of $500,000.

2.5 Assumptions and Dependencies

The Waverly Pictures Website assumes that all 20 San Diego theaters will continue to operate within the same centralized database using the Pacific Time Zone. It is assumed that a reliable database containing information about movies, showtimes, seating, prices, reviews, tickets, and loyalty accounts/subscriptions will be available. It is also assumed that the payment methods supported will be Visa, PayPal, and Apple Pay only. The system depends on the database structure remaining compatible with the requirements of the website. It also depends on the availability of a working server, internet connection, and supported web browser in order to operate correctly and according to the system requirements. Changes to the database structure, payment methods, server, internet connection, or supported web browsers may require changes to the SRS.

3. Specific Requirements

3.1 External Interface Requirements

Describes how Waverly Pictures will interact with external components, including user interface, physical devices, third party software programs and communication networks

3.1.1 User Interfaces

The platform is strictly a website accessed via a web address (URL) and does not use a mobile app. It is designed to be compatible with any operating systems like Google Chrome or other modern web browsers on both laptops and phones. The website includes web pages, interactive seat maps, and checkout screens. The seat map will use simple colors to show availability, such as green for open seats and red for taken seats

3.1.2 Hardware Interfaces

Customers will either use a laptop or phone connected to the internet to visit the website and navigate the pages using a mouse, keyboard, or touchscreen. At the entrance, a handheld theater scanner is used by staff to scan QR codes directly from the customer's device screen

3.1.3 Software Interfaces

The platform connects to external programs to manage daily operations. It links with payment gateways to safely process sales. It also uses SMS text message distributors to send text confirmations. In addition, it connects with the theater's central movie database to keep showtimes and seat maps accurate.

3.1.4 Communications Interfaces

Defines how the website sends data across the internet and ensures all communication is completely safe and reliable. The application relies on domain routing, so users can type a standard web address into their browser to open the website. To ensure user traffic, the website uses HTTPS (Hypertext Transfer Protocol Secure). This is necessary to transfer data safely between the user's browser and the website servers. It encrypts all sensitive data such as user passwords, personal info, and checkout details. The website also requires an SSL/TLS certification, which acts as a digital security certificate or an official ID badge for the platform. It proves that the website is safe and legitimate to the browsers. For notifications, the system must connect to external mail (SMTP) and text message servers using secure, encrypted ports. This ensures that when the website sends digital QR code to the user's email or phone, the data cannot be stolen or copied. This setup handles all external E-mail and SMS Data delivery safely.

3.2 Functional Requirements

3.2.1 <User Account>

3.2.1.1 Introduction
Users should be able to create and access their accounts through the website. The accounts should allow users to view previous purchase history and view any membership perks.

3.2.1.2 Inputs
The system accepts customer information to sign up or log in to the account in the form of a username or email as well as a password.

3.2.1.3 Processing
The system verifies the users' username/email and password with account information stored in the database. The system should also grant the user access to their account once logged in.

3.2.1.4 Outputs
The system outputs the account information like purchase history and membership information when the user logs in.

3.2.1.5 Error Handling
If the user provides an un-databased username/email, the system will notify the user that there is no account found and prompt the user to create an account.

3.2.2 <Movie Browsing>

3.2.2.1 Introduction
The system allows users to browse through a list of available movies, showtimes, and locations.

3.2.2.2 Inputs
The system accepts the input of a movie through the search function.

3.2.2.3 Processing
The system searches the movie database for available movies that correspond to the user's search input.

3.2.2.4 Outputs
The system displays a list of available movies that match the user's search. It should display the movie's name, showtimes, location and review score.

3.2.2.5 Error Handling
If there are no matching search results, the system should inform the user that there are no matching results.

3.2.3 <Movie/Ticket Selection>
3.2.3.1 Introduction
The system should allow users to select movie seats from a chosen movie showing temporary reservations.

3.2.3.2 Inputs
The system will take the customers' selected movies, showtime, and theater.

3.2.3.3 Processing
The system will take the requested movie and allow the user to select available seats for the respective movie showing. The system should also verify that the user has not picked over 5 tickets in a single purchase. The selected seats should be temporarily reserved for the user for 10 minutes for the purchase.

3.2.3.4 Outputs
The system will display the chosen movie, showtime, theater, and seats chosen. The system should also update the ticket availability in the system during the 5-minute reservation period and when the tickets are purchased.

3.2.3.5 Error Handling
If the tickets selected are no longer available for purchase, the system should prompt the user to pick new seats. If the 10-minute reservation period is up, the system should release the reserved tickets for other users to purchase. If the movie is 10 minutes past showtime, the tickets should become unavailable for purchase.

3.2.4 <Ticket Purchase>

3.2.4.1 Introduction
The system should allow users to purchase selected movie tickets with their supported payment methods.

3.2.4.2 Inputs
The system should take the users selected tickets, discounts, membership information and payment methods (Visa, PayPal, and Apple Pay).

3.2.4.3 Processing
The system should calculate the total ticket price of the chosen tickets with any discounts. It should also process the user's payment using the selected payment method and send confirmation to the user when the transaction is completed. The system should update the database with the purchased tickets and remove the chosen seats from future purchases.

3.2.4.4 Outputs
The system should output the completed purchase with a receipt. The system should also add the ticket purchase information to the user's membership account purchase history.

3.2.4.5 Error Handling
If the payment does not go through, the system should notify the user and allow them to choose a different method of payment or retry. If the 10-minute temporary ticket reservation expires, the tickets should be released, and the purchase should be canceled.

3.2.5 <Loyalty Account>
3.2.4.1 Introduction
The system should allow users to create and log into loyalty accounts. The accounts should store the user's personal information, payment information, purchase history, and loyalty points.

3.2.5.2 Inputs
The system should accept the user's information for their loyalty account. The system should also accept information about the users who previously purchased tickets for loyalty points.

3.2.5.3 Processing
The system account should store the user's loyalty account information and purchase history. The system should update loyalty points when acquired and/or used.

3.2.5.4 Outputs
The system should display the user's loyalty account information, purchase history, and usable loyalty points.

3.2.5.5 Error Handling
If the system does not have loyalty account information on file, the system should prompt the user to sign up for a new account. If the user chooses not to sign up for the loyalty account, the system should continue to function normally with no loyalty account connected to the user's account.

3.2.6 <User Feedback>

3.2.6.1 Introduction

The system should allow users to provide feedback after purchasing a ticket using a range of selectable smiley faces.

3.2.6.2 Inputs

The user selects a smiley face rating after purchasing a ticket.

3.2.6.3 Processing

The system records the users' selected ratings in its database.

3.2.6.4 Outputs

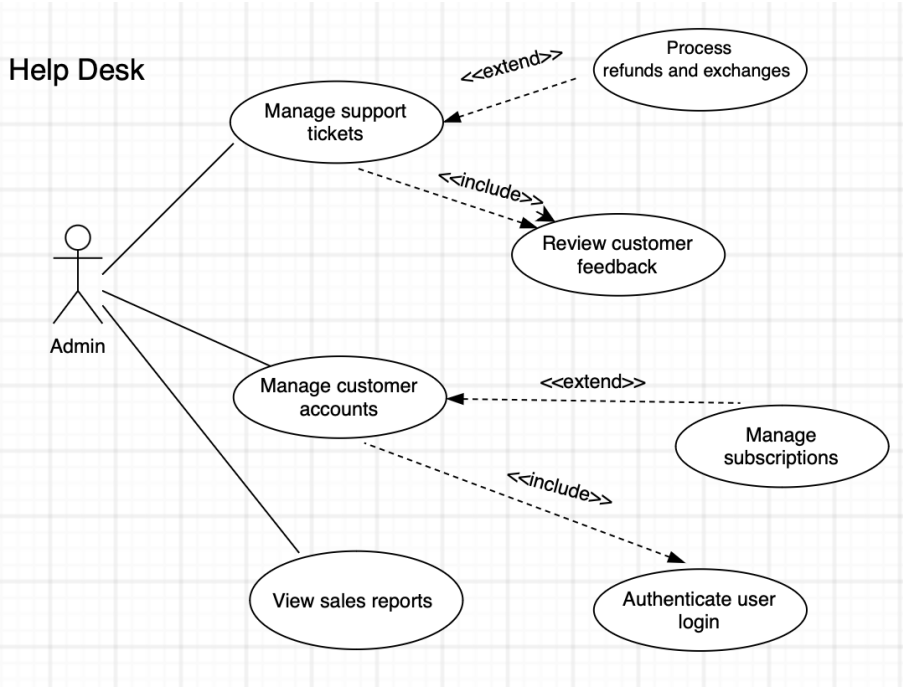
The system confirms to user that their response has been submitted.

3.2.6.5 Error Handling

If the user chooses not to complete the feedback, the system should still complete the ticket purchase but not record any feedback for that associated purchase.

3.3 Use Cases

3.3.1 Use Case #1



Use Case 1	Details
Name	Manage Support Tickets

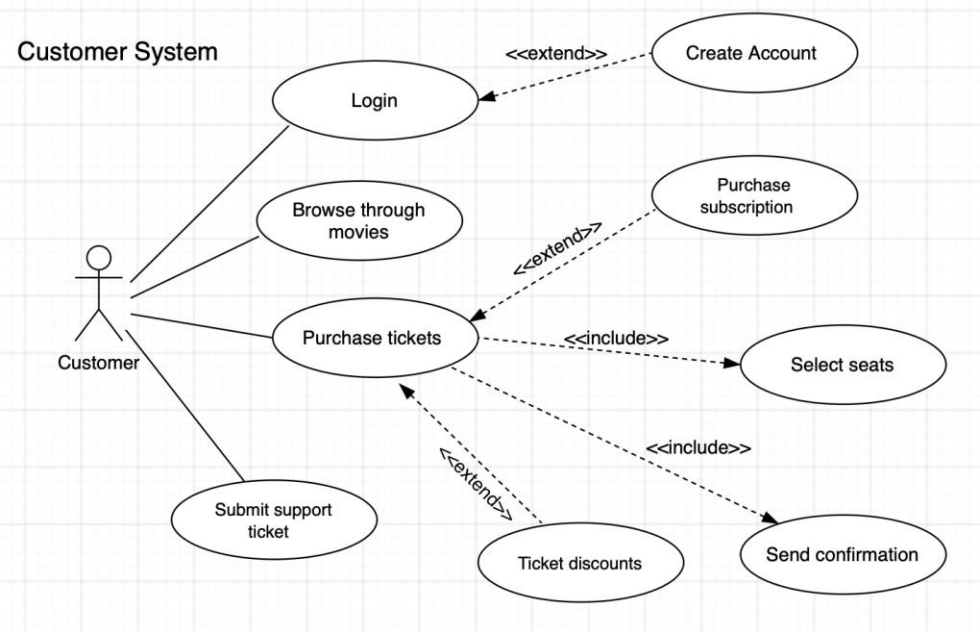
Actor	Admin
Description	Allows admin to view, respond to and resolve customer support inquires and ticket issues
Preconditions	Admin must be logged into the administrative dashboard
Includes	Review customer feedback
Extends	Process refunds and exchanges
Main Flow	1. Admin navigate the help desk module 2. Admins selects an open support ticket 3. Admin reviews ticket details and customer feedback 4. Admin provides a resolution or processes a refund/exchange 5. Admin marks the ticket as closed

Use Case 2	Details
Name	Manage Customer Accounts
Actor	Admin
Description	Admin can view user account details, update credentials, handle access issues and adjust subscription
Preconditions	Admin must be authorized
Includes	Authenticate user login
Extends	Manage subscriptions
Main Flow	1. Admin access the customer account directory 2. Admins search for and selects customer profile 3. Admin verifies user identity

	4. Admin updates account information, manages loyalty subscription or resets credentials

Use Case 3	Details
Name	View Sales reports
Actor	Admin
Description	Displays tickets sales data and revenue totals such as seats counts per movies, daily purchase logs and transaction records)
Preconditions	Admin must be logged into an authorized manager profile
Main Flow	1. Admin selects the reporting module 2. Admins choose a data range or specific theater location 3. Systems displays transaction logs and summary revenue statistics

3.3.2 Use Case #2



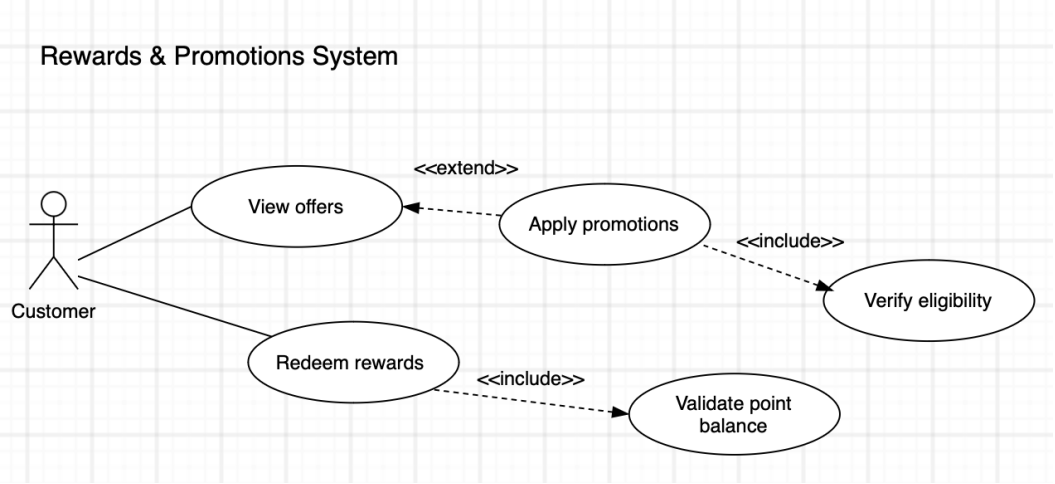
Use Case 1	Details
Name	Purchase Tickets
Actor	Customer
Description	Allows a customer to select movie showtimes, pick sets and complete ticket purchase
Preconditions	Customer is browsing available movie showtimes
Includes	Select seats, Send confirmation
Extends	Purchase subscription, Ticket discounts
Main Flow	1. Customer selects a movie showtime 2. Customers choose available seats. 3. Customer can opt in a subscription during checkout 4. Customer can apply student, military or senior discount at checkout 5. Customers enter payment details and complete purchase 6. Systems process payment and send booking confirmation

Use Case 2	Details
Name	User profile
Actor	Customer
Description	Customers can login, register a new account, and manage account details or subscription statues
Preconditions	Customer opens the Waverly Picture application
Extends	Create Account

Main Flow	1. Customer logs into their account or selects "Create Account” 2. Customers enter their username and password 3. System verifies credentials against the database 4. Systems log the user in and display their account dashboard
Alternative Flow	New Account: If the customer does not have an account, they select the option to register, complete sign-up form and then return to log in page

Use Case 3	Details
Name	Submit support ticket
Actor	Customer
Description	Customers can send inquires or refund request directly to Admin support
Preconditions	Customer opens the help section
Main Flow	1. Customer enters issue details and submits a ticket 2. System forwards the support ticket to admin queue

3.3.3 Use Case #3



3.4 Classes / Objects

3.4.1 <UserAccount>

3.4.1.1 Attributes

AccountInfo
username:String password:String name: String emailAddress: String UserProfile: Profile
membershipInformation: String purchaseHistory: Ticket [] paymentInformation: String

3.4.1.2 <Functions>

3.4.1.2 Functions

UserFunctions
newUser: NewUser myTicket:Ticket subscriptionPrice: double userInfo: UserInfo transactionID: int
login(): bool logout(): void createAccount(newUser: NewUser): bool

viewPurchaseHistory: void
viewMembershipInformation: void
viewPaymentInformation:void
securedPayment(ticket: Ticket): int
enrollsubscription(): void

3.4.2 <Movie>

Movie

MovieName : String
Showtime: Showtime
MovieRuntime : Double
MovieScore : Double
ReviewQuote : List<String>
TicketPrice : Double

displayMovieInformation(): List<String>
displayMovieShowtime(Showtime): String
displayMovieScore(): int
displayMovieCriticQuote(): String
displayMovieTicketPrice(): Double
searchMovie(String): List<Movie>

3.4.3 <Ticket>

Ticket

TicketInformation: List<String>
TicketIdentification: String
Movie: Movie
Showtime: Showtime
Theater: Theater
TicketType: String
Seats: Map<bool>
TicketPrice: Int
TicketStatus: Int
TicketAmount: Int

buyTicket(): void
getFreeSeats(): Map<bool>

selectSeat(index): bool
selectTicketQuantity(TicketAmount) : void
selectTicketDiscount(TicketType) : bool
verifyTicketLimit() : bool
generateTicketIdentification(Ticket) : Int

3.4.4 <LoyaltyAccount>

LoyaltyAccount

AccountInformation : List<String>
PaymentInformation : List<String>
LoyaltyPoints : Int
MembershipCard: String
MembershipStatus : String
RewardInformation : List<String>

createLoyaltyAccount(AccountInfo) : void
registerMembershipCard(String):
addLoyaltyPoints(AccountInfo) : void
redeemLoyaltyPoints(int): bool
displayAccountInformation(AccountInfo) : String
displayRewardInformation(AccountInfo) : String

3.4.5 <Feedback>

Feedback

Rating : Double
Purchase : Payment

displayFeedbackOptions() : List<int>
submitFeedback(int) : void
skipFeedback(): void
confirmFeedbackSubmission() : String

3.4.6 <Payment>

Payment

Tickets : List<Ticket>
PaymentMethod : String
TotalAmount : Double

PaymentCurrency : String PaymentStatus : bool Receipt : String PaymentReservationTimer: int
calculatePaymentTotal(Ticket) : Double convertCurrency(string) : Double processPayment(): bool retryPayment(): bool cancelPurchase(): void startReservationTimer(): void sendPaymentConfirmation(): void

3.4.7 <Theater>

Theater

TheaterID : Int TheaterSeatCapacity : Int TheaterType : String TheaterSeatMap : Map<bool> TheaterLocation: String Movie : Movie
getTheaterType(): String displayTheaterSeatMap() : String updateSeatAvailability(): void displayTheaterLocation(): String

3.4.8 <Showtime>

Showtime

Movie : Movie Theater: Theater Date: String Time: double AvailableSeats: Map<bool> PurchaseWindow: bool
displayShowtime() : String checkPurchaseWindow(): bool closePurchaseWindow() : void updateTicketAvailability(Ticket) : bool

3.5 Non-Functional Requirements

This requirement specifies how the website developed by Waverly Pictures should behave, how secure and available it should be. This is different from functional requirements in that it describes the quality criteria at the level of the system and includes measurable metrics for testing.

3.5.1 Performance

3.5.1.1 Concurrent Users: The system should be able to support at least 1,000 simultaneous users without experiencing any delay and be scalable enough to expand to handle 10 million users.

3.5.1.2 Loading Time: At least 95% of standard pages (such as movie searches, show times, and seat maps) should load completely within 2 seconds of a standard internet connection.

3.5.1.3 Database Querying Time: 99% of all database queries from any of the 20 movie theaters located in San Diego should process within 100 milliseconds.

3.5.1.4 Updating Time: As soon as one user selects a seat, the seat map should update within 1 second for all other users to prevent any duplicate selections by two users at once.

3.5.2 Reliability

3.5.2.1 System MTBF: The website should operate reliably with an MTBF of at least 30 days (720 operational hours) until a serious failure of the system.

3.5.2.2 Correctness of Transactions: All transactions and tickets should proceed without any hitches. In case the user's web browser fails midway in the transaction, the database must retain data integrity and not produce two tickets.

3.5.2.3 Accuracy of Hold Timer Function: The function must be accurate every time it is invoked. In cases where a user does not buy the tickets within 10 minutes, those seats must become available to everyone else within 2 seconds of the hold timer expiring.

3.5.3 Availability

3.5.3.1 Availability of Website: The website must have an availability of 99.9% (24/7), which means that there cannot be more than 8.76 hours of unplanned downtime on the servers within one year.

3.5.3.2 Periods of Maintenance: Maintenance must take place only during off-peak hours (Mondays from 2:00 to 4:00 AM Pacific Standard Time) and must not exceed 2 hours a month.

3.5.3.3 Recovery Point Objective (RPO): In case the database fails, no data shall be lost at all for ticket sales which have already been completed, and at most 10 minutes of the basic account log activity will be lost.

3.5.4 Security

3.5.4.1 Encrypted Connection: All connections to the website shall be established via HTTPS protocol (TLS 1.3) and must contain an SSL/TLS certificate so that there will be no hacker attacks.

3.5.4.2 Data Storage Encryption: The user's data will be stored in the database and encrypted. Passwords shall be hashed by bcrypt/Argon2 algorithm.

3.5.4.3 One Computer Usage: There should be a limit to how many computers can be used for one account. The second login will close all the other sessions automatically.

3.5.4.4 Protection against Botting: There should be a mechanism in place to prevent botting/scalping tickets to popular movies with a 99.9% success rate.

3.5.5 Maintainability

3.5.5.1 Mean Time to Repair (MTTR): In case of a big bug or site down in the production environment, the developers must be able to detect, repair, and redeploy the code within four hours max.

3.5.5.2 Code Design: The code should be well-designed with clean HTML, CSS, and Javascript. User interface code must not be mixed with backend code that calculates ticket pricing and discounts and seat logic.

3.5.5.3 Software Updates and Testing: Bug fixes and software updates will happen every two weeks, and testing for at least 80% of the most important functional code should happen.

3.5.6 Portability

3.5.6.1 Web Browsers: The website must look and work correctly on the two most recent versions of major web browsers, including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.

3.5.6.2 Screen Resolutions: The website interface must adjust smoothly on modern laptop and desktop screen resolutions (from 1280 720 up to 3840 2160 pixels) without breaking the layout or forcing horizontal scrolling.

3.5.6.3 Languages & Currencies: The application must allow users to switch between English, Spanish, and Swedish languages, and automatically convert ticket prices to the local currency in under 1 second.

Software Design Specification

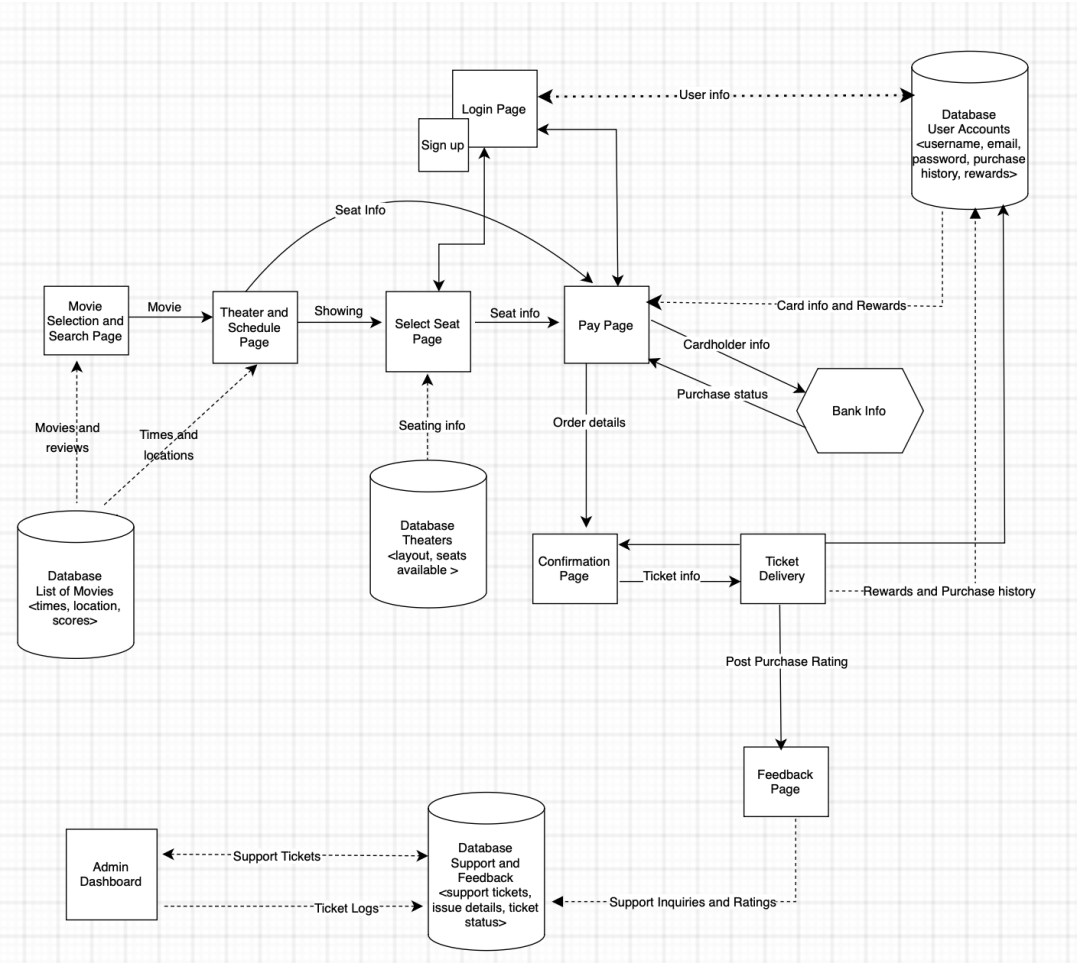
SWA

The Waverly Pictures web platform helps customer browse movies, choose showtimes, pick seats and buy tickets all online, the architecture is organized into three main parts:

1. Web Pages (Boxes): The screen that users see and interact with.
2. Database (Cylinders): Where information like showtimes, user accounts, and seat layouts are safely stored.
3. External Services (Hexagon): The bank system that processes card payments.

Diagram Key:

- Solid Lines: show the step-by-step path a user takes from screen to screen on the website
- Dashed Lines: show the backend data being saved or loaded from the database



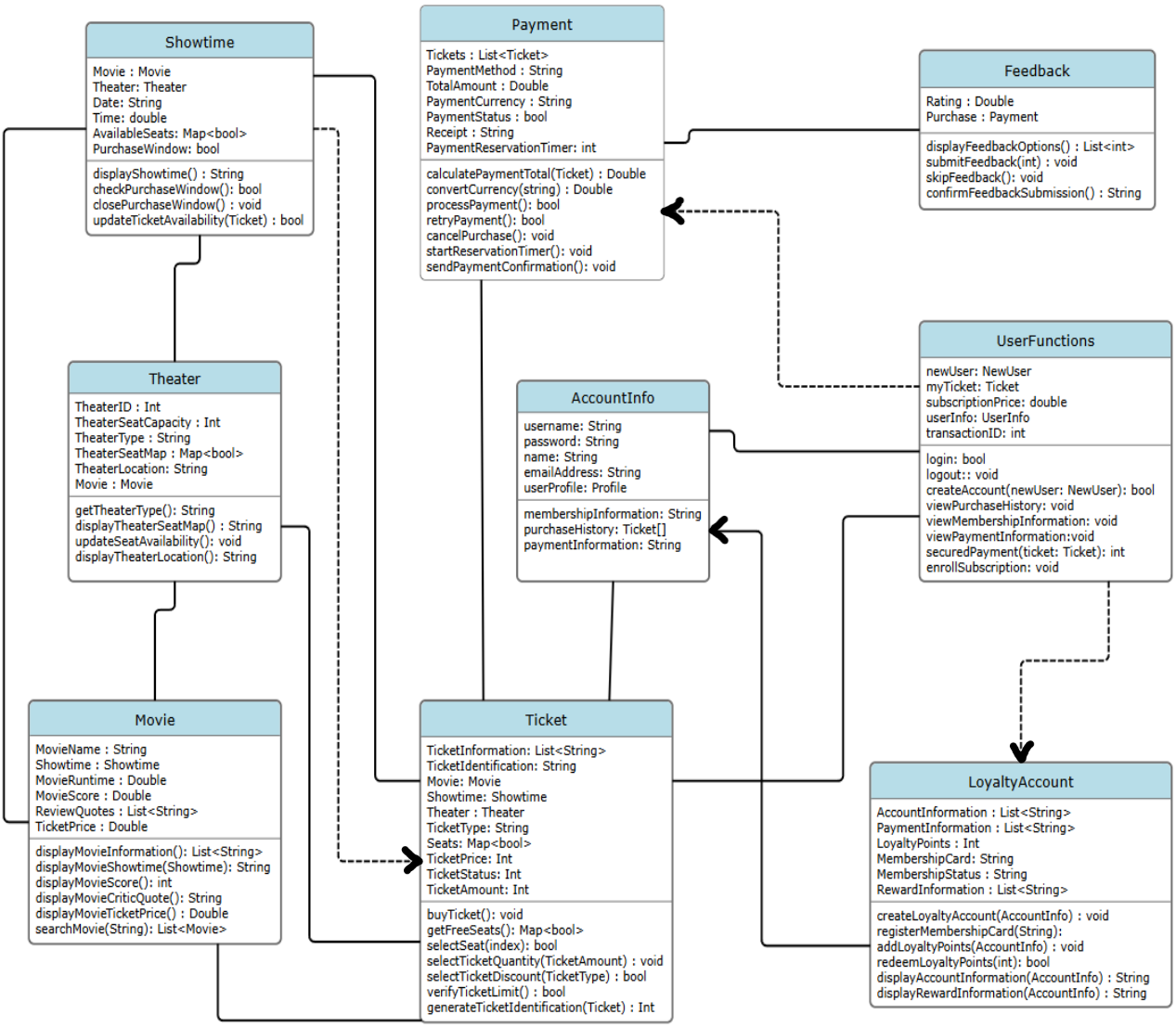
User & Transaction Flow

A customer begins browsing titles on the Movie Selection and Search Page, chooses a showtime on the Theater and Schedule Page, selects their seating on the Select Seat Page, which checks real time availability in Database Theaters. Before paying, the system routes the user through the Login/Sign Up Page to verify account details in Database User Accounts while holding their chosen seats. Once authenticated, the user proceeds to the Pay Page. After payment verification via Bank Info, the user proceeds to the Confirmation Page to Ticket Delivery, which saves transaction logs and updates rewards points.

Admin & Support Operations

Following ticket delivery, customers can visit Feedback Page to submit ratings, which are saved directly into Database Support and Feedback. Administrators access the Admin Dashboard to view support tickets, track ticket logs, and monitor platform activity.

UML Class Diagram



UML Class Description

The UML class diagram represents an online movie ticketing system. It outlines the system's core classes, detailing their attributes, operations, and how they relate to one another.

Class: AccountInfo
AccountInfo class is the main entity that contains information on customer profiles, credentials for accounts, payments, and transaction history. The properties of the class are username (String) – credentials that are used to sign in to the account, password (String) – encrypted password hashes, name (String) – customer's full name, and emailAddress (String) – address where

receipts and notifications will be sent. The other properties are: userProfile (Profile) – user preferences, membershipInformation (String) – subscription status, purchaseHistory (Ticket[]) – transaction history, and paymentInformation (String) – encrypted billing tokens. There are no methods in this class since the class works only as a container of data, and is managed by the UserFunctions controller class.

Class: UserFunctions
The UserFunctions class serves as the main controller class responsible for implementing user interaction flow control, session handling, user authentication, and order history maintenance. Some of the important attributes in this class include newUser(NewUser), myTicket(Ticket), subscriptionPrice(double), userInfo(UserInfo), and transactionID(int). This class supports functions like login() : boolean that authenticates the user credentials, logout() : void that logs out users from their sessions, createAccount(newUser : NewUser) : boolean that creates new user accounts, and viewPurchaseHistory() : void that retrieves the purchase history of the users. Other functions supported by this class are viewMembershipInformation() : void, viewPaymentInformation() : void, securedPayment(ticket : Ticket) : int that starts encrypted payment process, and enrollSubscription() : void.

Class: Movie
The Movie class represents the data model of each individual film entity within the system catalog, holding relevant data such as the metadata, review ratings, critic quotes, and pricing information. The attributes include movieName (String) that represents the official name of the movie, showtime (Showtime) representing the film's screening schedule, movieRuntime (double) representing the film length in minutes, movieScore (double) for review rating aggregation, reviewQuotes (List) for critic quotes of the film, and ticketPrice (double) for the movie ticket price. Methods include displayMovieInformation(): List<String> for formatting movie data output, displayMovieShowtime(s: Showtime): String for schedule output formatting, displayMovieScore(): int for converting rating into percentage, displayMovieCriticQuote(): String for retrieving featured quotes, displayMovieTicketPrice(): double for getting the base ticket price, and searchMovie(query: String): List<Movie> for querying movie database catalog.

Class: Ticket
The Ticket class implements individual and multiple ticket reservation by managing seat reservation, admission category selection, ticket discount calculation, and barcode generation. It contains such attributes as ticketInformation (List) with text information on ticket, ticketIdentification (String) representing unique QR code string, movie (Movie) – the selected movie, showtime (Showtime) - the time of the showing, theater (Theater) - the theater where the showing will take place, ticketType (String) – admission level (Adult, Student, Military, Senior), seats (Map) – the reservation of seats according to their coordinates, ticketPrice (int) – the price after discount application, ticketStatus (int) - status of the order (Pending, Reserved, Paid, Cancelled), and ticketAmount (int) with a purpose of imposing a limit on purchasing of 5 tickets in one transaction. Main methods include buyTicket(): void (order submission), getFreeSeats(): Map<bool> (availability check), selectSeat(index: int): boolean (seat selection switching), selectTicketQuantity(ticketAmount: int): void (setting of ticket quantity), selectTicketDiscount(ticketType: String): boolean (discount check and applying),

verifyTicketLimit(): boolean (imposing of purchase limits) and generateTicketIdentification(t: Ticket): int (barcode generation).

Class: Payment
The Payment class oversees transactions management, timing of seat reservations, currency conversion, and generation of receipts. Its attributes include tickets (List) representing the tickets included in an order, paymentMethod (String) to indicate payment gateway choice (e.g., Visa, PayPal, Apple Pay), totalAmount (double) representing the cost of transactions, paymentCurrency (String) to specify the code of the current currency, paymentStatus (boolean) indicating if checkout was successful, receipt (String) containing the receipt content, and paymentReservationTimer (int) representing the duration of the 10-minute (600 seconds) timer of the seat hold period. The class contains the methods which can be used for calculating grand total, currency conversion within the application, external gateway processing, payment retries, transaction cancellation, seat release, starting the reservation timer, and sending automatic receipt via email/SMS with QR codes: calculatePaymentTotal(ticket: Ticket): double, convertCurrency(targetCurrency: String): double, processPayment(): boolean, retryPayment(): boolean, cancelPurchase(): void, startReservationTimer(): void, and sendPaymentConfirmation(): void.

Class: LoyaltyAccount
The LoyaltyAccount class handles the loyalty accounts of the users of the theatre's loyalty rewards program. The LoyaltyAccount class holds accumulated loyalty points and rewards, perks, and other membership information. The LoyaltyAccount class data members include accountInformation (List) to hold information of connected accounts, paymentInformation (List) to hold information about saved billing profiles in auto-renewal services, loyaltyPoints (int) to hold information about reward points balance, membershipCard (String) to hold digital membership card ids, membershipStatus (String) to hold account status (Active, Expired, Suspended), and rewardInformation (List) to hold potential rewards. The operations performed by the LoyaltyAccount class include createLoyaltyAccount(accountInfo: AccountInfo): void to create loyalty accounts, registerMembershipCard(cardNum: String): void to register digital membership cards, addLoyaltyPoints(accountInfo: AccountInfo): void to award points, redeemLoyaltyPoints(points: int): boolean to use point discounts at checkout, displayAccountInformation(accountInfo: AccountInfo): String to return account information and displayRewardInformation(accountInfo: AccountInfo): String to format reward list.

Class: Theater
The Theater class encapsulates the physical locations of theaters in San Diego theater branches and handles issues of capacity, screen classification, and seat maps in real time. Attributes in the class include theaterID (int) which is the auditorium number, theaterSeatCapacity (int) representing the limit on the number of seats (75 for deluxe auditorium and 150 for normal), theaterType (String) representing the classification of the theater either deluxe or normal, theaterSeatMap (Map) to represent the availability of seats that are available or not, theaterLocation (String) for the theater branch location and movie (Movie) for the movie being shown.

Class: Showtime
The Showtime class models screening schedules and enforces business rules governing active ticket purchasing windows. Its attributes include movie (Movie) for the screening film, theater (Theater) for the assigned auditorium, date (String) for the scheduled calendar date, time (double) for the start time in 24-hour format, availableSeats (Map) for real-time seat availability maps, and purchaseWindow (boolean) to indicate whether ticket purchases are currently open. This class offers displayShowtime(): String, which formats the information about the schedule for display purposes, checkPurchaseWindow(): boolean, which checks whether the purchase takes place between 2 weeks before the movie and 10 minutes after the showtime; closePurchaseWindow(): void, which stops the purchase window in case the movie starts more than 10 minutes ago; and updateTicketAvailability(ticket: Ticket): boolean, which recalculates the availability of tickets.

Class: Feedback
The Feedback class keeps records of ratings made by customers after their purchases. The main features in this class are rating (double) that contains the numeric values of selected smiley faces and purchase (Payment) that relates feedback class with the purchase class. Some of the methods of this class include displayFeedbackOptions(): List<int>, which displays the feedback options to the customer; submitFeedback(rating: int): void, which stores the ratings made by the customer in the database; skipFeedback(): void, which allows the customer to skip making the feedback but not stopping the print-out of the receipts; and lastly, confirmFeedbackSubmission(): String, which confirms submission of the feedback.

Other Information:

Development Plan and Timeline

The development plan will include five modules:

Module 1 - User Interface and Presentation module: Template design using HTML/CSS, Browser UI design and seat map design using client-side Java Script.

Module 2 - User authentication and loyalty program module: Design of sign-up process for users, User authentication, Session locking and Loyalty rewards calculation.

Module 3 - Catalog, Search and reservation module: Design of movie catalog engine, Search for movie show times, Locking of seat reservations for 10 minutes and Ticket restriction

Module 4 - Payment Gateway and notification module: Integration with third party payment gateway services such as Visa, PayPal, Apple Pay, Foreign currency transaction, Discounts and receipts via email/SMS and QR code.

Module 5 - Database and help desk module: Schema partitioning of theater in San Diego, Query optimization for 100 milliseconds, Help Desk tickets and sales report.

Team Member Responsibilities Allocation

Team Member	Primary Roles & Modules	Assigned Responsibilities
Ows Zoabel	System Architect & Backend Lead	Design of database schema and partitions in San Diego locations, application controllers (UserFunctions, AccountInfo, LoyaltyAccount), security of sessions, and GitHub repositories.
Leah Bernstein	Frontend & UI Developer	Web presentation layer design (HTML/CSS/JavaScript), interactive seating chart for selecting auditorium seats (Theater, Showtime), and adaptive browser layouts for desktop displays.
Hsa Say	Integration, Business Logic & Testing Lead	Ticket processing and hold timer functions (Ticket, Payment), payment gateway connectors and currency exchange functionality, admin help desk reports (Helpdesk, Feedback), and project testing/QA.

Development Timeline (4 Month Project)

Month 1: Requirements Collection & Architectural Design

[W1-W2] SRS Requirement Specification & Scope Definition Completed
[W3-W4] DDS, Software Architecture & Database Design Completed

Month 2: Development of Major Front and Back-End Engine

[W5-W6] Development of HTML/CSS design & User Authentication/Account management module completed
[W7-W8] Development of Movie Search Engine, Theaters/Show Times mapping and Seat Interactivity engine completed

Month 3: Development of Business logic and APIs integration

[W9-W10] Development of Seats timer locking, 5 seats limit & Discounts module completed
[W11-W12] Development of Payment Gateway, Currency Conversion and SMS/Email gateway completed

Month 4: Development of Admin Panel, Testing & Delivery

[W13-W14] Development of Admin Reporting, Support Tickets Queue & Feedback module completed
[W15] System Testing, load testing (concurrent 1,000+ users) and Mean Time Between Failure verification performed
[W16] System Verification and Delivery of Documentation (in PDF form) on GitHub

SDS Test Plan:

Test Cases									
TestCaseId	**Component**	**Priority**	**Description/Test Summary**	**Pre-requisites**	**Test Steps**	**Expected Result**	**Actual Result**	**Status**	**Test Executed By**
SiteNavigation_1	Search_Bar_Module	P0	Verify that a user can navigate to Waverly Pictures website using a web browser and load the home page	Browser is launched	1. Write the url - http://waverlypictures.com in the browser's URL bar 2. Press enter.	Waverly Pictures website loads successfully and displays the home page	Homepage loaded completely upon navgating to http://waverlypictures.com	Pass	TesterK
PaymentProcess_2	Payment_Gateway_Module	P1	The discount amount should be calculated accurately during the time of payment, and the payment should be processed through the API of the payment gateway in 10 minutes.	User has selected tickets and navigated to the payment page.	1.Open payment page. 2.Select payment method (e.g., Visa) and enter billing info. 3.Apply discount code/type. 4.Click "Confirm Payment" before timer expires.	Payment is successfully authorized, reservation timer stops, and receipt is generated.	Payment processed successfully and transaction approved.	Pass	TesterA
LoginLogout_3	User_Auth_Module	P0	Make sure that the users can login securely using correct credentials and end their session when they logout.	Registered user account exists in database.	1. Navigate to Login page. 2. Enter valid username and password. 3. Click "Sign In". 4. Verify login and click "Logout".	User is authenticated upon login and session token is invalidated upon logout.	Authenticated successfully and logged out cleanly.	Pass	TesterA
SelectMovies_4	Catalog_Search_Module	P0	Users should be able to search or browse available films, gather information, and search for title/genre.	Movie catalog database contains active listings.	1. Navigate to movie catalog. 2. Enter query in search bar or filter by category. 3. Select a movie card to view details.	Matching movie list is displayed with accurate ratings, runtimes, and ticket prices.	Movie details displayed accurately.	Pass	TesterB
SelectShowTimes_5	Showtime_Schedule_Module	P1	Check that when a user clicks on a movie showtime, the ticket buying window opens up and the steaing map shows which seats are open or taken	User is on the movie schedule page and sees aviable showtimes	1. Click on a specific showtime slot 2. Watch for the tickt purchase window to open 3. Look at the theater seat map	The purchase window opens right away, showing the correct showtime details and an updated seat maps with avaiable seats	Purchase window opened and seating map loaded correctly	Pass	TesterK
SubmitFeedback_6	Feedback_Support_Module	P2	Verify that users ratings, review comments, and IT support inquiries are successfully submitted and logged	User is logged in and is on the Feedback Page	1. Go to Feedback & Support Page 2. Rate your experience and type your review comments 3. Fill in IT support request box with your issue details 4. Click the Submit button	Both the rating and IT support inquiry submit successfully and save to the Support and Feedback Database	Submitted rating (5 stars), review text, and IT support #1042; confirmed entry saved in Database Support and Feedback with ticket status as Open	Pass	TesterK

Confirmation_7	Ticket_Delivery_Module	P0	Verify order receipt of purchase and ticket details are correctly displayed on the Confirmation Page after payment	User completed a transaction on the Pay Page	1. Complete a ticket purchase 2. Wait for automatic redirection to the Confirmation Page 3. Check displayed receipt details (movie name, seats, order total, ticket ID)	The receipt order displays accurate ticket info and sent for delivery	Order receipt #9822 (ticket details) loaded correctly on Confirmation page; ticket info routed to delivery	Pass	TesterK
SelectSeat_8	Seat_Reservation_Module	P0	Verify that a user can select available seats from the seat map, then that the selected seats are held for the user for the durration of the reservation timer.	User is logged in and has selected a movie and showtime	1. Open the seat selection page for the event 2. Click on an available seat to select 3. Click to continue to the payment page	The selected seats are held as unavailable to other guests while the user goes to the payment page.	The selected seats are held for 10 minutes and the user is able to get to the payment page for the selected seats. When the page is loaded by another user, the selected seats are unable to purchase.	Pass	TesterA
UserAccount_9	User_Account_Module	P1	Verify that a user can update their account details and view correct previous purchase history	User is logged in and has at least 1 previous ticket purchase	1. Open the user account page 2. Edit account information (edit name and phone number) and click Save 3. Refresh the page and log in with new information 4. Check the previous booking history page	The profile is updated with the new name and phone number after refreshing the page. The booking history is also concurrent with all past purchases with their correct information	Profile is updated successfully with past booking history displayed correctly	Pass	TesterB
AdminDashboard_10	Admin_Dashboard_Module	P2	Verify that an admin can add new bookable events with different seat layouts and pricings.	User is logged in with an Admin account	1. Open the Admin dashboard page 2. Click Add event 3. Create new event with an event name, date and time, theater type, price, and seating layout. 4. Publish event and log out of Admin account 5. Log in as a regular user and search for the created event and purchase ticket	The event is saved and shown in the admin event list with the correct published status, and the correct seat map and prices for regular users	Event was published and is able to be viewed by a regular user account with correct name, date and time, theater type, price, and seating layout.	Pass	TesterK

Data Management

Data management strategy, including diagram and description.
Discussion and justification for design choices, including technology (SQL vs. non-SQL) and organization (multiple databases, table contents, etc.).

SQL vs non-SQL (Hsa): All of your software systems revolve around persistent and potentially sensitive data. Here you should specify your data management strategy, SQL or non. (Hint: diagrams always help)

Database (Leah): You should also discuss your design decisions: how many databases you chose and why, how you split up the data logically, possible alternatives you could have used (both in technology and organization of data)

Tradeoff Discussion (Ows): Justification of design decisions, including discussion of possible alternatives and tradeoffs.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2